

Chapter 3

METHODOLOGY

3.1 Introduction

This chapter describes the methodology used to develop the Marrybrown Sales and Payment Analytics Platform. The project combined reverse engineering of vendor-managed POS reports, a 1:1 replication and ELT workflow into a company-managed reporting database, and report-level parity validation through iterative comparison and reconciliation. Because the underlying vendor report logic was not fully documented, the methodology prioritised incremental delivery, repeatable validation cycles, staged deployment, and retention of evidence artefacts that could support traceability across analysis, design, implementation, and testing activities.

3.2 Methodology Choice and Justification

The project adopted an iterative and incremental development approach aligned with Agile principles, where functionality is delivered in small increments and continuously validated against expected outputs (Beck et al., 2001; Schwaber & Sutherland, 2020). This choice is suitable for report reconstruction because requirements, calculation edge cases, data-boundary issues, and workflow refinements emerge progressively during parity validation, and the cost of rework is reduced when changes are applied in smaller validated iterations (Pressman & Maxim, 2014; Sommerville, 2015). The same approach also supported gradual stabilisation across the replication layer, semantic/API layer, and portal layer, rather than treating the system as a one-pass build. The academic rationale for iterative delivery, ELT, replication-first traceability, and reconciliation-driven validation is synthesised in Chapter 2 (see Table 2.3).

3.3 Phases within the Iterative and Incremental Development Methodology

Work was organised as an iterative and incremental cycle that repeated for each targeted report module and supporting platform function. As illustrated in Figure 3.1, each cycle moved through six recurring phases: requirement analysis, system design, implementation, testing and validation, deployment, and review and feedback, before returning to requirement analysis when additional refinement was required. The phases were structured to preserve traceability from requirement interpretation to implementation and validation evidence, while allowing supporting operational functions such as controlled sync, quality checking, and scheduled automation to be introduced after core report logic had been sufficiently stabilised. Table 3.1 summarises the typical inputs, activities, and evidence artefacts produced in each phase.

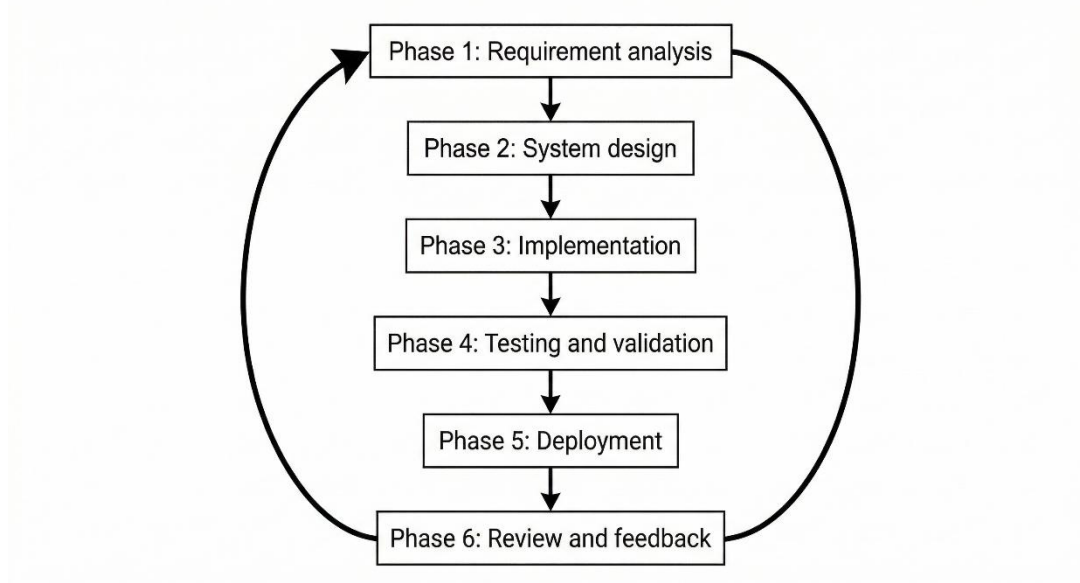


Figure 3.1: Iterative Workflow for Analytics Platform Development

Phase	Typical inputs	Key activities	Outputs / evidence artefacts (examples)
Phase 1: Requirement analysis	Vendor portal report views, exported files, parameter settings, stakeholder clarifications	Identify parameters and expected breakdowns; define output fields and totals; catalogue edge cases; draft data-to-report mapping; define acceptance criteria	Parameter matrix; output-field inventory; baseline export pack; acceptance checklist
Phase 2: System design	Phase 1 artefacts, replicated schema, constraints and non-functional requirements	Specify replication scope and join paths; define refresh boundary and rerun handling; define API contract and export behaviour; define validation plan and test cases	Report specification draft; API contract; data mapping; test plan; design diagram references
Phase 3: Implementation	Design artefacts, replicated datasets, acceptance criteria	Implement version-controlled report queries and rules; build API endpoint; implement parameter validation and formatting; add	Endpoint implementation; reproducible queries; sample outputs; execution logs

		logging and error handling	
Phase 4: Testing and validation	Baseline exports, platform outputs, acceptance checklist	Compare outputs; investigate discrepancies; refine rules; run regression checks; record acceptance decisions and reconciliation results	Comparison evidence; discrepancy log; updated rule versions; validation records
Phase 5: Deployment	Validated module, environment-ready checklist	Deploy validated changes; configure access controls; update release documentation; prepare controlled operational use	Deployed endpoint or portal view; deployment checklist; release notes
Phase 6: Review and feedback	Deployed module, stakeholder feedback	Capture usability gaps and missing cases; triage defects versus enhancements; update acceptance criteria and test scenarios	Feedback log; change requests; revised acceptance criteria

Table 3.1: Phase-level inputs, activities, and evidence artefacts (per report module)

The phase descriptions in Sections 3.3.1 to 3.3.6 elaborate the responsibilities summarised in Table 3.1 and explain how each phase contributed to parity-driven report reconstruction and evidence retention.

3.3.1 Phase 1: Requirement Analysis (Document Analysis and Stakeholder Feedback)

Requirements were elicited through document analysis of vendor portal reports and exported files, together with stakeholder clarification from Finance and Operations users regarding business intent, expected breakdowns, and acceptance expectations for continuity reporting. Because vendor logic was treated as a black box, the project applied a reverse engineering perspective to infer business rules from observable inputs and outputs (Chikofsky & Cross, 1990). Black-box testing principles supported this phase by enforcing comparable parameters such as outlet and date range for subsequent parity checks (Myers et al., 2011). In practical terms, this phase produced a parameter matrix, an output-field inventory, and a baseline export pack that served as the reference set for later comparisons. Edge cases that could materially affect parity, such as voids, returns, split tenders, and rounding behaviour, were recorded as test scenarios to support repeatable validation as rules evolved.

Key outputs of Phase 1 therefore included a report-specific parameter list, an output-field inventory, a draft data-to-report mapping, and an acceptance checklist describing what constituted an acceptable parity match for the report. These artefacts provided the basis for both implementation planning and validation evidence retention in later phases.

3.3.2 Phase 2: System Design

System design translated Phase 1 requirements into design artefacts across the platform layers. At the replication and data-preparation level, this included determining the minimum replication boundary for each targeted report by tracing required output columns and business rules back to the relevant source tables, join paths, and reference data. The replicated schema preserves a 1:1 table structure with the source database, but only the tables required for the targeted reports are selected for replication. In the project context, transactional data replication is generally bounded from 2025 onward in line with organisational storage constraints, whereas

reference tables are replicated in full. The design also defined refresh boundaries, rerun considerations, and the operational approach for controlled portal-triggered sync, source-to-replica quality checks, and scheduled ETL functions.

At the semantic/API and portal layers, the design specified both the report contract and the report presentation in a traceable manner. This included endpoint naming, required and optional parameters, response schema, output columns, totals and subtotals, sorting and formatting rules, and export behaviour aligned to vendor report structures. Report design also defined the use of reusable shared components and a common layout across the portal, while allowing report-specific configuration for parameters, column sets, and output behaviour. The design further established a validation plan by mapping acceptance criteria and edge-case scenarios to concrete test cases and evidence artefacts such as baseline exports and comparison outputs. Design modelling artefacts such as architecture diagrams, data flow, and simplified data model views are presented in Chapter 4, while detailed per-report specifications are consolidated in Appendix A.

3.3.3 Phase 3: Implementation

Implementation followed a logic-first sequence: report logic was developed at the semantic/API layer and exercised using controlled parameters aligned to Phase 1 baseline exports. Where feasible, query prototypes were first validated against the replicated schema to confirm data availability, join correctness, and boundary handling, after which logic was stabilised into version-controlled queries and exposed through an API endpoint with a defined request and response contract. Implementation work also included consistent parameter validation, deterministic output ordering where required for comparison, and export-ready formatting so that parity checks could be performed reliably against vendor exports. Logging and error handling were incorporated to support traceability of runs during iterative validation cycles.

During early iterations, controlled manual sync and rerun procedures were used to stabilise replicated data windows and report logic. After the replication flow and

validation approach had matured, the platform was extended with scheduled ETL capabilities, administrative automation controls, and data-quality checking support so that the validated reporting environment could support both portal-triggered sync operations and managed automation.

3.3.4 Phase 4: Testing and Validation (Parity and Reconciliation)

Testing and validation were performed at both module and system levels. At the module level, parity validation was conducted by generating platform outputs using identical parameters to vendor exports and comparing results at the appropriate granularity, including row-level comparison where applicable and aggregate-level comparison where exports grouped outputs. Comparisons were performed using consistent filtering and boundary rules, and discrepancies were investigated and resolved through iterative refinement of filters, joins, date handling, and edge-case treatment. To reduce regression risk, previously validated scenarios, including recorded edge cases, were rechecked after rule changes so that fixes for one report did not introduce unintended drift in another.

At the system level, reconciliation checks were applied to confirm cross-report consistency, such as alignment between sales totals and payment totals for the same reporting windows under defined rules. Source-to-replica quality checks were also used for defined sync windows to confirm that replicated data remained fit for report reconstruction and investigation work. Together, these checks operationalised data quality perspectives such as accuracy, completeness, and consistency (International Organization for Standardization, 2008; Wang & Strong, 1996). Validation artefacts, including baseline exports, comparison outputs, and acceptance decisions, were retained to support repeatability.

The validation methodology also included black-box comparison against vendor outputs, white-box review of reconstructed business rules and query behaviour, and user-oriented verification of report retrieval, viewing, and export functions. In addition, a basic operational performance evaluation was used to assess usability under

typical usage conditions. Response-time observations for representative report queries were considered in relation to practical expectations for routine Finance and Operations workflows. The intent of this evaluation was to confirm operational acceptability rather than to conduct exhaustive benchmarking or load testing.

3.3.5 Phase 5: Deployment

Upon meeting validation criteria for a module, the corresponding API and portal changes were deployed to the target environment according to an environment-ready checklist. Deployment activities included configuration validation such as database connectivity and access controls, release documentation to support traceability of rule changes, and rollback considerations for changes that materially affected report outputs. In this project, deployment was staged so that validated report modules and supporting platform functions could first be checked in a controlled environment and then consolidated into the broader implemented platform environment. This phase also covered the operationalisation of controlled sync procedures, scheduled ETL functions, and related administrative controls for report operations.

3.3.6 Phase 6: Review and Feedback

Feedback was collected from Finance and Operations users to identify usability issues, missing data elements, and any residual discrepancies observed during real workflows. Feedback was documented as traceable change requests and triaged into parity defects that required rule correction and enhancements that adjusted workflows or presentation without changing core financial meaning. Where feedback changed acceptance criteria or revealed new edge cases, the iteration returned to Phase 1 for the affected module so that requirements artefacts could be updated and the revised logic validated again with retained evidence.

As shown in Figure 3.2, the high-level operational workflow of the implemented platform began with data extraction and replication from the external POS environment, followed by ELT processing into the company-owned SQL Server reporting database, source-to-replica quality checking, semantic/API-based report reconstruction, and final report delivery through the internal reporting portal to Finance and Operations users. This view links the development phases described above to the end-to-end technical flow implemented and validated during the project.

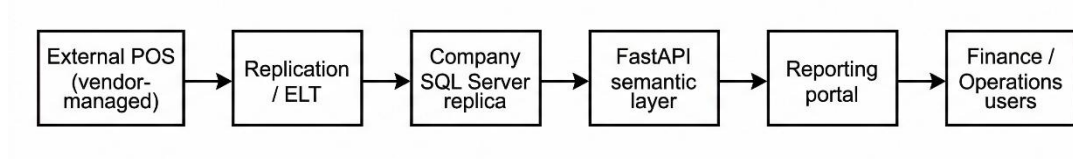


Figure 3.2: High-level system workflow (data replication to report delivery)

3.4 Project Schedule (Gantt Plan for the 40-Week Internship Period)

For planning and execution purposes, the overall project was organised across a 40-week internship period. Because the project followed an iterative and incremental methodology, the schedule was not treated as a strictly linear one-pass sequence in which all analysis ended before all design, and all design ended before all implementations. Instead, the work was arranged as overlapping blocks in which early planning activities established the foundation, while later report reconstruction, validation, deployment, and refinement cycles revisited earlier findings as new report behaviours and operational issues emerged. Table 3.2 therefore presents a detailed week-based Gantt-style plan that reflects both sequencing and overlap across the internship period.

Work package	Weeks (planned)	Main focus	Relation to the iterative and incremental methodology
Project initiation and company briefing	1-2	Understand business context, reporting	Establishes the initial problem boundary

		dependency, internship scope, and stakeholder expectations	before the first iteration cycle begins
Problem formulation and scope definition	2-4	Define the reporting problem, project boundary, and high-level objectives	Produces the baseline scope used for early iterations, while allowing later refinement
Literature review and source collection	3-8	Review references on vendor dependency, replication, ELT, validation, and reporting portals	Builds the academic basis that informs repeated design and validation decisions
Methodology definition and project planning	5-8	Define the iterative and incremental approach, report-validation method, and initial work structure	Formalises the cycle used repeatedly for report reconstruction modules
Current system study and report requirement analysis	6-12	Analyse vendor portal behaviour, report parameters, output structure, and acceptance expectations	Supplies the first iteration backlog and is revisited when new edge cases appear
System analysis and high-level design	9-14	Prepare architecture, data-flow, and interface design foundations	Creates the initial technical blueprint for implementation iterations
Replication scope definition and schema preparation	11-16	Identify required source tables, define replication boundary,	Supports the first implementation increment by making

		and prepare the reporting database	the required data available
Initial replication and ETL workflow setup	14-18	Establish controlled extraction, loading, and manual rerun procedures	Provides the first usable data-refresh process for early report iterations
Iteration Cycle 1: early report reconstruction	16-20	Reconstruct selected reports at backend level and perform first parity checks	First implementation increment using the full analysis-design-build-validate loop
Iteration Cycle 2: rule refinement and additional reports	19-24	Extend report logic coverage, resolve first-wave discrepancies, and improve parity	Second increment that builds on feedback from the first validated modules
Portal workflow development and shared UI integration	20-26	Implement report search, parameter input, tabular viewing, and export workflows	Frontend functions are integrated in parallel with backend report increments
Iteration Cycle 3: expanded report reconstruction and regression checking	23-28	Add more report modules, recheck earlier modules, and reduce rule drift	Demonstrates repeated incremental extension with regression-aware validation
Controlled deployment and environment consolidation	25-30	Deploy validated modules, verify access control, and stabilise environment setup	Moves validated increments into controlled operational use
Source-to-replica quality checking and sync refinement	28-33	Strengthen portal-triggered sync support, quality re-	Extends the methodology beyond report logic into data-

		checking, and investigation workflow	quality assurance iterations
Scheduled ETL and administrative automation refinement	30-35	Introduce and refine scheduled ETL behaviour and related administrative controls	Represents a later operational increment after core parity work is sufficiently stable
Iteration Cycle 4: operational refinement and final validation pass	32-37	Revisit report behaviour, operational edge cases, and remaining mismatches	Final major refinement cycle before closure and handover preparation
Documentation, handover preparation, and final report consolidation	35-40	Consolidate technical documents, handover materials, and academic report inputs	Captures the outputs of all prior iterations into final closure artefacts

Table 3.2: Detailed Gantt-style project schedule across the 40-week internship period

3.5 Implementation Environment Requirements

The platform environment was specified at a practical level to support replication workloads, concurrent report queries, validation work, and controlled operational use. Table 3.3 summarises the high-level hardware considerations, while Table 3.4 summarises the software components and their roles within the platform. Together, these tables indicate that the project environment was designed around operational sufficiency for report reconstruction, testing, and delivery rather than around large-scale enterprise optimisation.

3.5.1 Hardware Requirements

At the hardware level, the platform required sufficient compute, memory, storage, network stability, and environment availability to support replication, API serving, and stakeholder validation activities. As summarised in Table 3.3, the hardware considerations were framed around dependable execution of replication and ELT tasks, responsive report generation, and stable access for iterative stakeholder review.

Resource	Requirement (high-level)	Justification
Compute	Multi-core CPU capacity	Supports replication and ELT tasks together with concurrent API request handling.
Memory	Sufficient RAM for database and service workloads	Supports SQL Server caching and in-memory processing during report generation.
Storage	Low-latency storage with growth headroom	Supports replicated tables, indexes, logs, and export artefacts with predictable query performance.
Network	Stable connectivity between source, replica, and services	Required for extraction, loading, and portal access; affects refresh time and report responsiveness.
Availability	Environment availability aligned to validation and operational use	Supports iterative validation and reduces disruption during reporting-intensive windows.

Table 3.3: Hardware requirements (high-level)

3.5.2 Software Requirements

At the software level, the platform depended on a relational database, a Python-based ELT environment, an API framework, a frontend framework, and version control tooling. As summarised in Table 3.4, these components collectively supported

the end-to-end workflow of data replication, report logic reconstruction, portal-based report delivery, and traceable change management.

Component	Role in the platform	Notes
Microsoft SQL Server	Hosts the replicated transactional schema	Provides relational storage for replicated tables and report queries.
Python	Implements replication, ELT scripts, and supporting utilities	Supports iterative development and operational scripting for extraction and loading.
SQLAlchemy + pyodbc	Database connectivity from Python	Provides a maintainable access layer to SQL Server via ODBC.
Microsoft ODBC Driver 17/18	ODBC connectivity to SQL Server	Driver version depends on host environment configuration.
FastAPI	Semantic/API layer for report endpoints	Exposes reconstructed report logic as RESTful services (FastAPI, n.d.).
React + Node.js	Reporting portal frontend	Implements report parameter inputs, tabular display, and export workflow (React, n.d.).
Nginx / web serving layer	Portal delivery and API proxy support	Supports frontend delivery and controlled routing in the deployed environment.
Git	Version control for code and documentation	Supports traceability of changes to report logic and validation artefacts.

Table 3.4: Software requirements (high-level)

3.6 Summary

This chapter presented the iterative methodology used to reconstruct vendor-aligned report logic, execute replication and ELT processes, and validate report-level parity through comparison, reconciliation, and supporting operational checks. Chapter

4 presents the analysis and design artefacts that operationalise this methodology for the Marrybrown Sales and Payment Analytics Platform.